

Rapport Labo 2

Kenan Augsburg

07 November 2025

Contents

1. Code analysis	3
1.1. Global constants	3
1.2. main	3
1.2.1. load_or_generate_keys	3
1.2.1.1. generate_keypair	4
1.2.1.1.1. scalar_mult	4
1.2.1.1.1.1. is_on_curve	5
2. Errors	7
2.1. random.randrange	7
2.1.1. Bad range $\{0, \dots, N - 1\}$	8
2.1.2. Usage of random.randrange	9
2.2. load_or_generate_keys	9
2.2.1. Blind loading	9
2.2.2. Insecure storage of private key	10

1. Code analysis

In this section, we'll go over the global constants and each function in the order they are called to explain what they do and how/where they are used.

Note

The explanation is mainly given as comments in the code accompanied with the addition of typing whenever it makes sense

1.1. Global constants

At the start of the scripts, we have multiple constants defined which are the parameters for the secp256k1

```
1 # --- secp256k1 parameters ---
2 P = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEC2F
3 A = 0
4 B = 7
5 Gx
  = 55066263022277343669578718895168534326250603453777594175500187360389116729240
6 Gy
  = 32670510020758816978083085130507043184471273380659243275938904335757337482424
7 N = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141
```

En comparant avec les paramètres que l'on trouve en ligne¹ on peut confirmer que ces paramètres sont correctes.

1.2. main

This is the entry point of the program.

1.2.1. load_or_generate_keys

This function is called at the start of `main` with this snippet

```
1 priv, pub = load_or_generate_keys()
```

This is the only time this function is called.

```
1 def load_or_generate_keys() → tuple[int, tuple[int, int]]:
2     """This function loads an existing pair of private/public keys or
3     generates,
4     saves and return a new pair.
5     """
6     # Files containing the private and public keys
7     priv_file = "ecdsa_private.key"
8     pub_file = "ecdsa_public.key"
```

¹<https://neuromancer.sk/std/secg/secp256k1>

```

9
10     if os.path.exists(priv_file) and os.path.exists(pub_file):
11         # If both files exist, proceed to extract the private and public keys.
12         with open(priv_file, "r") as f:
13             # The private key is converted from a base16 string to an int.
14             # NOTE: The fact that we strip what we read shouldn't matter
15             # unless
16             # the key isn't stored properly
17             priv = int(f.read().strip(), 16)
18         with open(pub_file, "r") as f:
19             # The public key is read as a JSON object with the keys x and y
20             # containing integers in base64
21             pub_data = json.load(f)
22             pub = (int(pub_data["x"], 16), int(pub_data["y"], 16))
23         print("Loaded existing keypair from disk.")
24     else:
25         print("No keypair found – generating new one...")
26         # If any of the files do not exist we generate a new pair of keys
27         priv, pub = generate_keypair()
28         with open(priv_file, "w") as f:
29             # The private key is simply written properly in hex format
30             f.write(hex(priv))
31         with open(pub_file, "w") as f:
32             # The public key is saved correctly in JSON
33             json.dump({"x": hex(pub[0]), "y": hex(pub[1])}, f)
34         print("New keypair generated and saved.")
35     return priv, pub

```

In this function we call `generate_keypair` to generate a pair of keys.

1.2.1.1. `generate_keypair`

```

1 def generate_keypair() → tuple[int, tuple[int, int]]:
2     """Generate a pair of keys"""
3     # Take a random value between 1 and N (N not included)
4     priv = random.randrange(1, N)
5     # Multiply the private key with G to get the public key
6     pub = scalar_mult(priv, (Gx, Gy))
7     return priv, pub

```

 Python

We take a random number between 1 and N for the private key

$$a \in \{1, \dots, N - 1\} \quad (1)$$

And for the public key, we call `scalar_mult`

1.2.1.1.1. `scalar_mult`

```

1 def scalar_mult(k: int, point: tuple[int, int] | None):
2     """Multiply a point on the curve by the scalar value k"""

```

 Python

```

3     # Check that the point is on the curve otherwise the multiplication isn't
4     # possible.
5     assert is_on_curve(point)
6     if k % N == 0 or point is None:
7         # If k is perfectly divisible by N or now point was given, the result
8         # is None.
9         # NOTE: If k is perfectly divisible by N, the result will be a None,
10        # this check saves time
11        return None
12    if k < 0:
13        # If k is negative, we simply multiply both inputs by -1 which is
14        # simple to do and allows the next part of the algorithm to work.
15        return scalar_mult(-k, (point[0], (-point[1]) % P))
16    result = None
17    addend = point
18    # To calculate the result, we add point k times, effectively multiplying
19    # the point by k
20    while k:
21        if k & 1:
22            result = point_add(result, addend)
23            addend = point_add(addend, addend)
24            k >>= 1
25    return result

```

This function should work. As a sanity check we can check with sage that the result is as expected.

```

1  E = EllipticCurve(GF(P), [A,B])
2  G = E(Gx,Gy)
3
4  for i in range(100):
5      k = random.randrange(1,N)
6      assert(scalar_mult(k, (Gx,Gy)) == (k * G).xy())
7
8  for i in range(100):
9      k = -random.randrange(1,N)
10     assert(scalar_mult(k, (Gx,Gy)) == (k * G).xy())

```

sage

The only difference would be with the case of $k = 0$ since when done mathematically the result is $\mathcal{O}$ but since the program uses points with x and y coordinates it cannot represent this value. The choice of using `None` is valid but not that explicit. (Does every `None` represent $\mathcal{O}$ or is it an invalid value?)

1.2.1.1.1.1. `is_on_curve`

```

1  def is_on_curve(point: tuple[int, int]):
2      """Check if a point is on the Elliptic curve"""
3      if point is None:

```

Python

```

4         return True
5     x, y = point
6     # Weistrass curve  $y^2 = x^3 + ax + b$ 
7     # NOTE: This is mathematically sound
8     return (y * y - (x * x * x + A * x + B)) % P == 0

```

Given $k = (x, y)$ and $x, y \in \mathbb{Z}$, the function properly applies the Weistrass curve.

$$\begin{aligned}
 y^2 &= x^3 + ax + b \pmod{n} \\
 y^2 - (x^3 + ax + b) &= 0 \pmod{n}
 \end{aligned}
 \tag{2}$$

2. Errors

☰ Tâche 1

For each error:

- Show the snippet
- Explain what the problem is and the implications. What an attacker can do and what could go wrong.
- Explain in simple terms why this should be fixed
- Give a fix

2.1. `random.randrange`

This function is used twice in the code.

```
1 def generate_keypair() → tuple[int, tuple[int, int]]:
2     """Generate a pair of keys"""
3     priv = random.randrange(1, N)
4     pub = scalar_mult(priv, (Gx, Gy))
5     return priv, pub
```

Python

```
1 def sign_message(priv, message):
2     z = sha256(message)
3     while True:
4         k = random.randrange(0, N)
5         R = scalar_mult(k, (Gx, Gy))
6         r = R[0] % N
7         if r == 0:
8             continue
9         k_inv = inv_mod(k, N)
10        s = (k_inv * (z + r * priv)) % N
11        if s == 0:
12            continue
13        return (r, s)
```

The documentation² for the module and its function starts with a warning to not use it for security and cryptographic purposes. This kind of warning shouldn't be ignored since it means that the prngs it provides are either not proved to be cryptographically secure or worse, proved to be attackable.

Furthermore, the range can be a problem independently of how good the distribution is. When multiplying a point with a scalar value, if the value is equal to zero or a multiple of N the result will be $\mathcal{O}$ which can be problematic.

For the first usage, the range gives us a value $\text{priv} \in \{1, \dots, N - 1\}$. This means that we shouldn't get a `None` but just to be safe we should have an assertion before returning our keys.

²<https://docs.python.org/3/library/random.html>

For the second usage, we have $k \in \{0, \dots, N - 1\}$ which means that R can be `None` which will raise an exception when accessing `R[0]`. This means that there is a $\frac{1}{N}$ chance of having an issue when trying to sign a message.

We have two problems here.

2.1.1. Bad range $\{0, \dots, N - 1\}$

The problem here is that there is a chance of raising an exception and the implications vary depending on how exceptions are handled by the caller as well as which code paths lead to this function being executed.

The caller shouldn't expect `sign_message` to fail when given valid parameters. Depending on how they handle exceptions, the program could end up revealing informations which could be exploited or end up in an unrecoverable state.

If an attacker can trigger this function a lot, it could be used to leak secrets or for a denial of service attack.

If the exception results in an unrecoverable state such as the program exiting There is a serious risk of service degradation proportional to the usage.

In simple terms, the risk is that the service becomes unavailable or worse, that secrets could be leaked and used by bad actors to falsify prescriptions.

The simple fix is to change the range to $\{1, \dots, N - 1\}$. (Using an assertion as a sanity check and also allows the typing system to narrow after the multiplication)

```
1 def sign_message(priv, message):
2     z = sha256(message)
3     while True:
4         k = random.randrange(1, N)
5         R = scalar_mult(k, (Gx, Gy))
6         assert R
7         r = R[0] % N
8         if r == 0:
9             continue
10        k_inv = inv_mod(k, N)
11        s = (k_inv * (z + r * priv)) % N
12        if s == 0:
13            continue
14        return (r, s)
```

As stated previously, the range used in `generate_keypair` is correct but we should still have an assertion after the multiplication since it doesn't cost us much and having $\mathcal{O}$ as a public key would be catastrophic. (since $\forall k \in \mathbb{Z}, k\mathcal{O} = \mathcal{O}$)

```
1 def generate_keypair() → tuple[int, Point]:
2     priv = random.randrange(1, N)
3     pub = scalar_mult(priv, (Gx, Gy))
```

```
4     assert pub
5     return priv, pub
```

2.1.2. Usage of `random.randrange`

≡ Tâche 2

snippet

≡ Tâche 3

implications

≡ Tâche 4

attacker

≡ Tâche 5

what could go wrong

≡ Tâche 6

simple terms why fix

≡ Tâche 7

fix

2.2. `load_or_generate_keys`

2.2.1. Blind loading

```
1  if os.path.exists(priv_file) and os.path.exists(pub_file):
2      # If both files exist, proceed to extract the private and public keys.
3      with open(priv_file, "r") as f:
4          # The private key is converted from a base16 string to an int.
5          # NOTE: The fact that we strip what we read shouldn't matter unless
6          # the key isn't stored properly
7          priv = int(f.read().strip(), 16)
8      with open(pub_file, "r") as f:
9          # The public key is read as a JSON object with the keys x and y
10         # containing integers in base64
11         pub_data = json.load(f)
```

```

12     pub = (int(pub_data["x"], 16), int(pub_data["y"], 16))
13     print("Loaded existing keypair from disk.")
14     # WARN: if both files exist, their value is taken as ground truth. There
15     # is no verification that the private key is in [1;N] and that the
16     # public key is the result of priv * G

```

When loading the pair of keys from disk, there are no checks made to validate it. The problem is that to be able to properly sign and verify the signature of a message you need the private key a to respect $1 \leq a < N$ and the public key A should be $A = aG$.

☰ Tâche 8

implications

☰ Tâche 9

attacker

☰ Tâche 10

what could go wrong

☰ Tâche 11

simple terms why fix

☰ Tâche 12

fix

2.2.2. Insecure storage of private key

```

1  else:
2      print("No keypair found – generating new one...")
3      # If any of the files do not exist we generate a new pair of keys
4      priv, pub = generate_keypair()
5      with open(priv_file, "w") as f:
6          # The private key is simply written properly in hex format
7          f.write(hex(priv))
8      with open(pub_file, "w") as f:
9          # The public key is saved correctly in JSON
10         json.dump({"x": hex(pub[0]), "y": hex(pub[1])}, f)
11     print("New keypair generated and saved.")
12     # WARN: The public and private keys are both simply saved to disk

```

 Python

```
13 # without any specifications when it comes to permissions. (on linux,  
14 # resulted in 644 permissions)
```

When either a public key or private key file is missing, a new pair of keys is generated and saved to disk using the built-in open function in 'w' mode.

There are multiple implications here.

- Since the 'w' mode is used, any existing file is overwritten if it already exists. In the case where the private key exists but the public key isn't found, the private previous private key will be lost.
- The keys will have the default permissions (determined by the OS). On Linux, it often means 0644 which is an issue since we don't want any other user to be able to read the private key.

In the event of an attacker gaining access to the host running the program, they can extract the private key without any privilege escalation. This would allow them to falsify records. (Breaks authenticity and integrity)

Effectively, this would mean that the attacker could create a fake prescription and the pharmacy would treat it as valid. For doctors, this would allow the attacker to create fake symptoms for a patient which in turn would result in more work for the doctors to check and effectively destroy the efficiency brought by the system.

This issue should be fixed because it could cause serious legal troubles since it involves medical prescriptions. Furthermore, it also risks the loss of trust in the system.

☰ Tâche 13

fix